

Autoencoder Implementation on MNIST Dataset

Zahra Khodagholi

1 Introduction

This document covers the implementation of two types of autoencoders - Fully Connected and Convolutional - applied to the MNIST dataset of handwritten digits.

2 Code Overview

2.1 Dependencies

- `torch`: Neural network models and training.
- `torchvision`: Loading and transforming MNIST dataset.
- `torch.optim`: Optimization algorithms.
- `matplotlib`: Plotting and visualizing images.

2.2 Fully Connected Autoencoder

2.2.1 Architecture

Encoder with two linear layers (784 to 256, then 256 to 128 neurons) with ReLU activation, and a Decoder mirroring the Encoder, using Sigmoid activation in the final layer.

2.2.2 Forward Pass

Flattens input before passing through the Encoder and reshapes it back to 28x28 after the Decoder.

2.3 Convolutional Autoencoder

2.3.1 Architecture

Encoder with two convolutional layers followed by max-pooling layers, and a Decoder with three convolutional layers, first two followed by upsampling layers.

2.3.2 Forward Pass

Compresses and then reconstructs the image back to its original size.

2.4 Training and Evaluation

- Loss Function: Mean Squared Error (MSE).
- Optimizer: Adam (learning rate = $1e-3$).
- Epochs: 10.
- Visualization: Post-training, displays 20 images from the test set.

3 Observations and Analysis

3.1 Fully Connected Autoencoder

- Training Performance: Consistent improvement in loss.
- Parameters: 468,368 parameters.
- Reconstructed Images: Compare with original images for quality assessment.

3.2 Convolutional Autoencoder

- Training Performance: Stagnant loss, suggesting issues in learning or architecture.
- Parameters: 3,721 parameters, showcasing parameter efficiency.
- Reconstructed Images: Quality needs assessment due to stagnant loss.

3.3 Comparative Analysis

- Parameter Efficiency: Convolutional autoencoder is more parameter-efficient.
- Learning Efficiency: Fully connected autoencoder showed better learning in this implementation.
- Image Reconstruction: Qualitative assessment of reconstructed images is crucial.

3.4 Recommendations for Future Work

- Optimizing Convolutional Autoencoder: Investigate training stagnation.
- Adjusting Model Complexity: Explore varying the complexity based on reconstruction quality.

3.5 Output

3.5.1 Fully Connected Autoencoder

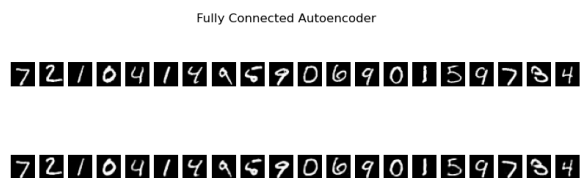


Figure 1: Reconstructed Images by the Fully Connected Autoencoder

3.5.2 Convolutional Autoencoder

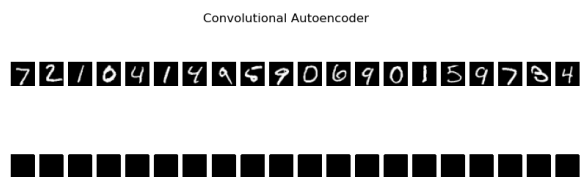


Figure 2: Reconstructed Images by the Convolutional Autoencoder

4 Conclusion

This document presented the implementation and results of Fully Connected and Convolutional Autoencoders on the MNIST dataset, highlighting the differences in architecture, parameter count, and reconstruction capability.